上海人工智能实验室
Shanghai Artificial Intelligence Laboratory

Technical Report

CELLPACK: EFFICIENT DATA LOADING FOR BILLION-CELL TRAINING

Zhongqi Wang [1 2] **Mingqian Ma** [1 3] **Yucheng Wu** [1 4] **Yizhou Wang** [2] **Dong Yuan** [5] **Yuchen Ren** [1]

ABSTRACT

Single-cell pretraining is expanding from tens of millions toward billions of cells. At this scale, training corpora exceed main memory, making efficient storage-backed random access essential for sustaining training throughput. Existing general-purpose formats face a trade-off between compression efficiency and access granularity: block compression amplifies reads and decoding for individual cells, while compressing each cell independently sacrifices compression efficiency. We present CellPack, an end-to-end data loading system that addresses this trade-off by sharing statistical models across independently decodable cell records. Its asynchronous loader overlaps reads, decoding, and model-input construction while preserving the requested sampling order. To further reduce I/O and repeated computation, we formulate caching over a known access sequence as a memory-constrained optimization problem and use a cost-aware planner to allocate memory between compressed pages and decoded cells. On 16 CPU cores, the loader delivers 1.165M cells/s from a single PCIe 4.0 NVMe SSD. Under fully random access, offline planning with a cache budget of 10% of the compressed corpus size reduces page misses by 38% and increases end-to-end loading throughput by 58% relative to the same loader without caching. On a billion-cell corpus constructed by replicating real biological data with independently permuted gene mappings, CellPack sustains an end-to-end loading throughput of 3.67M cells/s across five full shuffled epochs.

1 INTRODUCTION

Single-cell foundation models learn representations of genes and cells from large collections of gene expression measurements. These representations support tasks ranging from cell-type identification to predicting cellular responses to perturbations (Cui et al., 2024; Gandhi et al., 2025). The data takes the form of a cell-by-gene expression matrix, typically with around 10% non-zero entries (Gong et al., 2023). Such matrices are commonly stored in compressed sparse row (CSR) form, which retains each cell's non-zero values and gene indices (Virshup et al., 2024).

The scale of this pretraining is expanding rapidly. scGPT uses a 33M-cell training corpus (Cui et al., 2024); Tahoe-x1 reaches 266M training cells (Gandhi et al., 2025). Public repositories are larger still: scBaseCount contains over 502M cells from 27 organisms (Youngblut et al., 2026). With Arc Institute planning over one billion cell experiments for virtual-cell modeling (Arc Institute, 2026), data infrastructure must prepare for billion-cell pretraining.

This transition pushes training data beyond host memory. For the sparse expression profiles produced by 10x Chromium 3′ sequencing (Zheng et al., 2017), a CSR representation with 2,000 non-zero genes per cell and 32-bit values and indices occupies 16 KB per cell before compression. At this density, 266M cells require 4.26 TB, already exceeding the 2 TB of host memory in an NVIDIA DGX H200 system (NVIDIA, n.d.). A billion cells require 16 TB, making storage-backed access essential on such nodes. Training must repeatedly retrieve cells in the sampler's requested

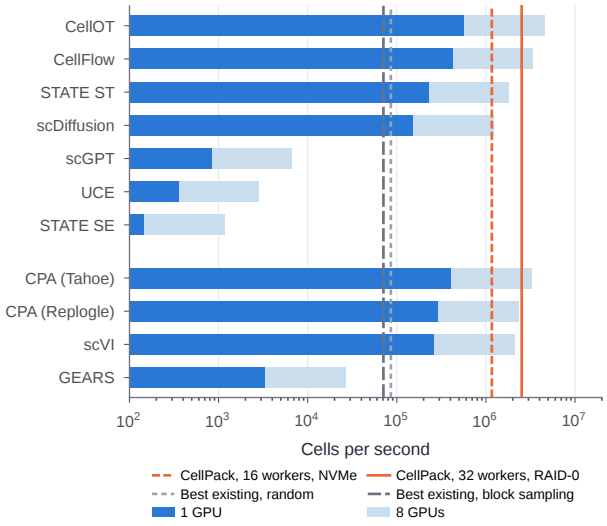


Figure 1. **Model demand and loader supply.** Bars show measured one- and eight-GPU demand. Vertical lines show loader throughput, below the demand of several models.

[1]Shanghai Artificial Intelligence Laboratory [2]Jilin University [3]Carnegie Mellon University [4]Fudan University [5]University of Sydney. Correspondence to: Yuchen Ren <renyuchen@mail.pjlab.org.cn>.

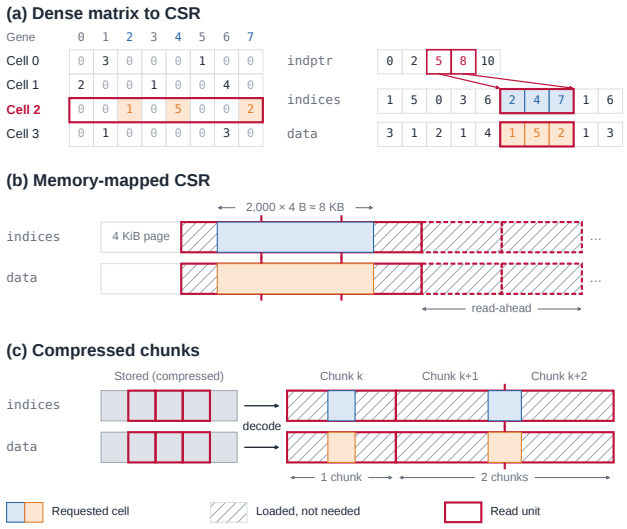


Figure 2. **Cell storage formats and loading methods.** (a) Sparse cell-by-gene matrices ($C \times G$, typically 10% density) commonly use CSR: `indptr` (cell offsets), `indices` (gene indices) and `data` (nonzero values). (b) Memory-mapped reads are page-granular; unaligned cell boundaries and random-access read-ahead fetch unrelated data. (c) Compressed chunks typically exceed one cell. Unaligned cells may span chunks, requiring every touched chunk to be loaded and decompressed in full.

order, including a fresh global permutation each epoch. Sustaining pretraining therefore requires efficient random access even when only a fraction of the corpus fits in memory.

Existing loaders already fall short of the data demand of several current single-cell models (Figure 1). CellFlow processes 423.6K cells/s on one H200, counting source and target cells, and STATE ST processes 227.6K cells/s. Across the evaluated configurations and cache budgets, the fastest existing random-access loader, Lance, supplies 85.8K cells/s; the block-sampling reference, annbatch, supplies 70.8K. The random-access rate is only one fifth of CellFlow's single-GPU demand. Data delivery already limits the use of available compute.

We present CellPack, a storage format and end-to-end loading system for billion-cell training beyond main memory. Our work makes three contributions.

A systematic analysis of single-cell loading bottlenecks. We trace storage reads, decoding and input construction across single-cell loaders and general-purpose formats to explain their throughput limits (Figure 3). Large compressed blocks save space but require reading and decoding unrelated cells; independent per-cell compression reduces this extra work but loses compression efficiency. Uncompressed storage avoids decoding but transfers more bytes per cell. These trade-offs motivate a format that combines compact storage with fine-grained access.

A compressed format with independent cell access. We separate the unit of statistical modeling from the unit of decoding. Cells share coding models that capture the structure of sparse expression data, while each cell remains independently decodable. Shared statistics improve compression without coupling the decoding of neighbouring cells. It achieves a 5.89× compression ratio in our evaluation while retaining direct access to individual cells.

An end-to-end loader for billion-cell training. We turn this format into a complete path from stored expression matrices to model-ready batches, including gene alignment and input construction, while preserving the trainer's sampling order. On 16 CPU cores, CellPack delivers 1.165M cells/s from a single PCIe 4.0 NVMe SSD without a data cache. On a billion-cell corpus constructed by replicating real biological data with independently permuted gene mappings, it sustains 3.67M cells/s across five full shuffled epochs.

2 Background and Motivation

2.1 Single-cell storage and data loading

Single-cell datasets pair expression matrices with cell and gene annotations. AnnData stores them in H5AD, backed by HDF5, or Zarr (Virshup et al., 2024; Zarr Developers, n.d.). CSR uses `indptr` to locate each cell's slices in the `indices` and `data` arrays (Figure 2). Chunking and compressing these arrays independently can place physical boundaries inside a cell.

A loader retrieves cells, aligns genes and constructs batches. AnnLoader and MappedCollection access on-disk AnnData rows (Virshup et al., 2024; Lamin Labs, n.d.); annbatch, scDataset and cellarium group reads (Gold et al., 2026; D'Ascenzo & Cultrera di Montesano, 2026; Cellarium AI, n.d.). BioNeMo uses memory-mapped sparse arrays (St. John et al., 2024), and TileDB-SOMA supports sparse-array queries (TileDB, Inc., n.d.). We also evaluate memory-mapped arrays, Arrow-backed HF Datasets, Parquet and Lance (Lhoest et al., 2021; The Apache Software Foundation, n.d.; Lance Developers, n.d.). Table 1 compares their storage modes on CELLxGENE-10M (Section 5.1).

2.2 Compression versus fine-grained access

A random cell request must read and decode the compressed units containing its indices and values. Unrequested neighbours consume bandwidth and decoding work unless they are reused before eviction. Compression can therefore save storage without making random reads cheaper.

Figure 3a measures device bytes read per delivered cell,

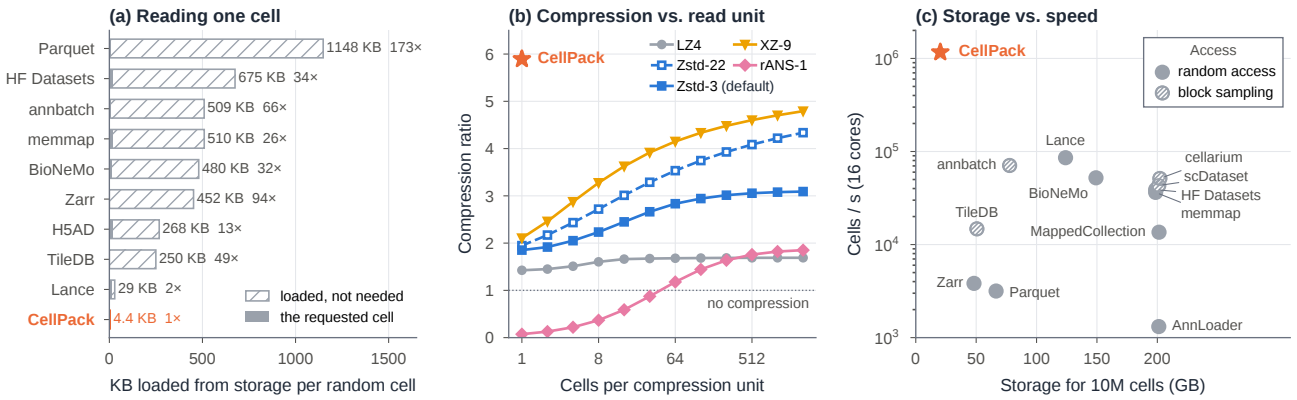


Figure 3. **Bytes read versus bytes needed for cell access.** (a) Measured device reads per delivered cell: solid bars mark average stored bytes; hatching marks additional traffic. Ratios use a 4 KiB denominator floor. (b) Compression ratio versus independently compressed unit size on fixed input data. (c) Corpus footprint versus each loader's best measured throughput on 16 cores; marker patterns distinguish random access from block sampling. CellPack uses no data cache in (a) and (c).

Table 1. **Loader storage features on real biological data.** Checks mark chunk access, compression and memory mapping. Size: kB/cell; read unit: average minimum access size. *Native block/chunk sampling.

Method	Chunk	Comp.	Mmap	Size	Read unit
Zarr	✓	✓		4.82	739 KB chunk
TileDB*	✓	✓		5.06	332 MB batch
Parquet	✓	✓		6.65	849 KB group
annbatch*	✓	✓		7.77	497 KB chunk
Lance	✓	✓		12.40	8 KiB block
scDataset*	✓			20.12	83 KB chunk
AnnLoader	✓			20.12	83 KB chunk
MappedCollection	✓			20.14	83 KB chunk
cellarium*	✓			20.21	202 MB shard
BioNeMo			✓	14.92	–
HF Datasets			✓	19.86	–
memmap			✓	19.86	–
CellPack		✓		2.04	4 KiB page

divided by average stored bytes per cell after compression, with a 4 KiB denominator floor. Measurements include startup and warm-up at each configuration's smallest runnable cache budget. They combine format granularity, page alignment, read-ahead, cache reuse and loader overhead. Parquet reads 1,148 KB per cell against 6.65 KB stored; Lance reads 29 KB against 12.40 KB. These ratios measure end-to-end read amplification, rather than compression-unit size alone.

Figure 3b encodes fixed inputs in independent units of 1–2,048 cells to isolate compression granularity. Larger units share more statistical context and amortise metadata; smaller units reduce unrelated decoding but compress less effectively. For k cells of s uncompressed bytes each and compression ratio $R(k)$, a cold single-cell request reads roughly $ks/R(k)$ bytes before page and metadata overhead. Better compression helps only insofar as it offsets the larger unit.

Figure 3c connects these costs to throughput. From MappedCollection through HF Datasets and BioNeMo to Lance, smaller representations accompany higher delivery rates, consistent with less data movement through storage and memory. Compact Zarr and Parquet remain slow because access and processing costs also matter. The resource measurements in Section 5.2 distinguish SSD, CPU and memory-bandwidth limits. Small units alleviate read amplification but store more bytes per cell; large units improve compression but fetch more unrelated data. CellPack targets compactness and independent cell access together.

2.3 Block sampling and training order

Locality can instead come from the sampler. annbatch fetches contiguous slices through `chunk_size` (Gold et al., 2026); scDataset shuffles blocks and mixes cells within multi-batch buffers (D'Ascenzo & Cultrera di Montesano, 2026); TileDB-SOMA-ML shuffles groups within larger I/O batches (TileDB-SOMA-ML Developers, n.d.). Nearby requests amortise reads and decoding, but larger blocks reduce mixing and larger buffers consume memory. Without a global preshuffle, batches can retain correlations among neighbouring cells from the same study, experimental batch or cell type. annbatch recommends preshuffling (Gold et al., 2026).

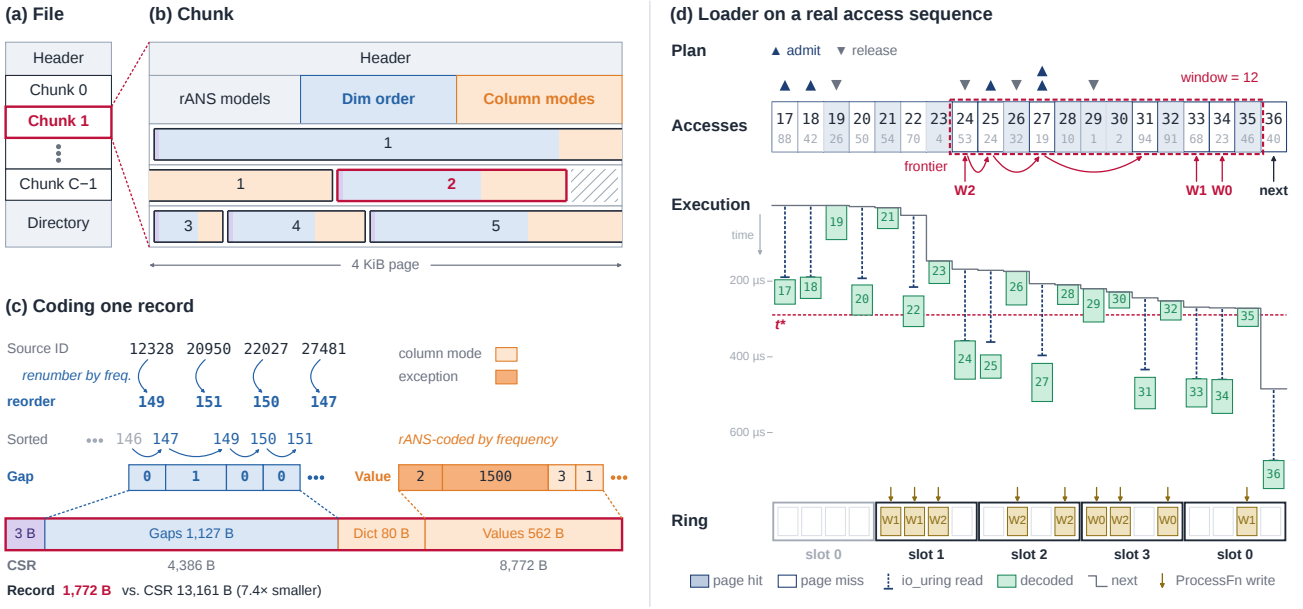


Figure 4. **The CellPack format and loader.** (a, b) File and chunk layout. (c) A record encoded from real data: rank 0 denotes the column mode, and exceptions use a local dictionary. Widths reflect code lengths; measured bytes are compared with CSR below. (d) Execution of a real access sequence. The earliest unfinished access bounds the execution window; workers overlap reads and decoding before writing batch-ring outputs.

Section 5.7 compares fresh global permutations (RR), blocks from a globally preshuffled sequence (BP), and annbatch sampling on the stored order (BR). BR produces pronounced training-loss excursions in scVI and CPA on Replogle, and persistently higher loss in STATE SE (Figure 8). Their final OOD losses are 0.84%, 0.18% and 1.56% higher than RR, respectively (Table 4); BP tracks RR closely. The effects depend on the model and dataset, but demonstrate that source-order locality can alter training stability and quality.

Physical preshuffling improves mixing by reading and rewriting the corpus. Repeating that order is straightforward; fresh global permutations require random reads or reorganisation. HF Datasets distinguishes index shuffling from materialisation through `flatten_indices()` (Hugging Face, n.d.). We target the trainer's chosen sequence without requiring a particular physical order or sampling scheme.

2.4 Accessing data with limited memory

Caching avoids repeated work when cells share pages or are revisited across epochs. Buffered reads and memory mappings use the OS page cache (The Linux Kernel Development Community, n.d.); loaders can also retain compressed or decoded data explicitly.

With equal-sized pages, equal miss costs and known future requests, Belady's optimal rule evicts the page whose next use lies furthest away (Belady, 1966). LRU instead uses past recency, a weak guide when each cell appears once per shuffled epoch. Training provides more information: the sampler can supply future accesses or regenerate them from a seed.

A compressed page saves a read and can serve several cells; a decoded row consumes more memory but also saves decompression. Minimising misses alone need not maximise throughput once CPU work or memory traffic becomes limiting. Memory allocation must therefore account for both representation size and the work avoided along the complete loading path.

3 Design

3.1 The CellPack format

A chunk supplies the statistical context for compression; a cell remains the unit of access and decoding (Figure 4a–c). Our default chunk holds 4,096 cells, shared coding models and a directory of independent records.

Encoding sparse indices and repeated values. Sorted gene indices have concentrated gaps, which we entropy-code after renumbering genes by their frequency within the chunk. The inverse mapping restores gene identities.

Expression counts also concentrate on small integers. Cell-wise scaling and log normalisation preserve equal values within a cell, so normalised floating-point rows retain substantial repetition. Each record stores distinct value bit patterns in a sorted, difference-coded dictionary and represents values by ranks. Chunk-level per-gene modes provide rank 0; exceptions use the local dictionary (Figure 4c). All value bits are preserved.

Gap, dictionary and rank streams use rANS with separate chunk-shared probability tables (Duda, 2014); all cells share one gap model. A record carries its local dictionary, counts, stream lengths and coding states. Decoding needs only that record and shared metadata, amortising model storage without coupling neighbouring cells.

Packing records into pages. A compressed record can still cross a page boundary. We pack records so that each touches its minimum $\lceil s/P \rceil$ pages, for record size s and page size P. Best-fit decreasing packs the remainders $s - (\lceil s/P \rceil - 1)P$ into page-sized bins, adding padding between bins. The directory preserves logical cell ids despite physical reordering. In a packing measurement with an average cell size of 2.02 kB, below the 4 KiB page size, padding reduced the compression ratio by about 7% but cut page reads by 23% relative to tight concatenation. Sub-page records then require one page; larger records touch only the pages needed for their encoded length.

3.2 An end-to-end loader over cell records

Access and reuse planning. The loader takes stored matrices, a cell sequence and an output specification. Directories resolve each request to its record's pages; after obtaining shared metadata, only those pages are fetched and that cell decoded. Gene alignment and input construction produce model-ready inputs. This removes compression-block amplification while retaining page-granular I/O.

Known future accesses enable offline caching: compressed pages save reads, while larger decoded rows also save decompression. Belady's rule supplies the equal-sized, equal-cost serial reference (Belady, 1966), with bypass allowed. Our practical approximation first ranks row-reuse intervals by saved work per occupied byte and reuse distance, $w/[s(j-i)]$, then plans pages for the remaining reads. A small search over budget splits selects the lowest estimated loading time. The compiler emits position-indexed admission and release actions, either for the full sequence or incrementally within an online lookahead.

Concurrent execution and ordered delivery. Workers overlap reads, decoding and input construction within a bounded execution window (Figure 4d). Concurrent requests share in-flight page reads. The plan reserves memory for this window and reclaims objects only after outstanding accesses can no longer use them. Workers may finish out of order, but shared-memory batch buffers deliver the requested sequence and are reused only after consumption. This integrates planned reuse with storage concurrency and training input construction; Section 5.4 measures its effect on both page misses and delivered throughput.

4 Implementation

The system uses Rust, Python bindings and scalar, AVX2 and AVX-512 kernels. Asynchronous `io_uring` reads overlap I/O and decoding (Axboe, 2019); NumPy and PyTorch views expose shared-memory batches. A single-host distributed wrapper assigns batches to training ranks, and converters accept H5AD files and AnnData objects.

5 Evaluation

We evaluate random-access throughput, compression and decoding cost, cache efficiency, and billion-cell scaling, then relate loading rates to model demand and examine how sampling order affects training.

5.1 Setup

Hardware. All experiments run on a server with Intel Xeon Platinum 8558 CPUs and NVIDIA H200 GPUs. Data is stored either on one PCIe 4.0 NVMe SSD or on a two-SSD RAID-0 array of the same model; the array is the default.

Data. The loader benchmark uses ten million cells resampled from 432 human datasets in CELLxGENE (CZI Cell Science Program et al., 2025), with sample counts approximately proportional to dataset size. Cells retain their source order. Compression experiments use the raw and normalised matrices of these datasets. The billion-cell experiment replicates the same data with independently permuted gene mappings (Section 5.5).

Loaders. The baselines comprise seven single-cell loaders and five general-purpose formats, with storage modes, footprints and read granularities listed in Table 1 and configurations in the supplementary material.

Workloads. The default access order is a uniform random permutation of all cells. We also test shuffled blocks of 256 and 4,096 cells, and dataset- and cell-type-balanced sampling, which draw a group uniformly and then a cell within it. Batches contain 256 cells with 2,048 `float32` gene values per cell.

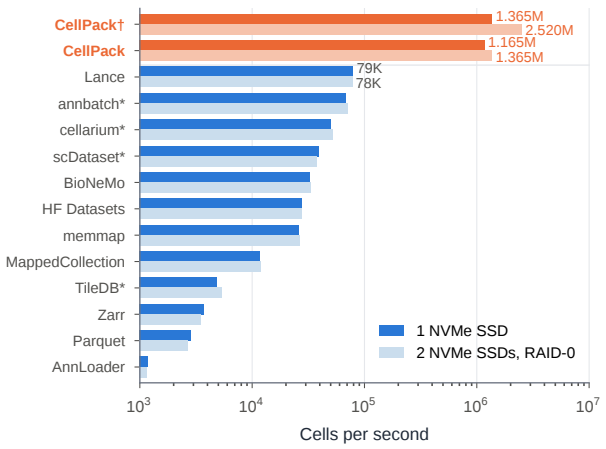


Figure 5. **Loader throughput on real biological data.** CellPack: 16 workers; CellPack†: 32 workers. Baselines: 16 workers and a 20 GB cache budget, separate from process RSS.

Resources and measurement. Runs use fixed physical-core sets and cgroup-controlled cache budgets. Baselines use 16 cores and 0–80 GB of OS page cache. CellPack uses 16 or 32 cores and cache budgets of 0, 20 and 80 GB, with direct I/O and explicit cache management; 20 GB approximately holds its compressed corpus. The default throughput metric averages a 60 s post-warm-up window.

CellPack retains 9,967,232 cells from 430 datasets, emitting 8,192 bytes per cell. No-cache points in Figure 5 are single 5 s checks after one warm-up epoch, without a variance estimate. Cached configurations retain the 60 s protocol. Loading, online planning and consumption share the core budget here; the cache-policy ablation isolates planner cores.

5.2 Loading throughput under controlled resources

Baselines under random order. Figure 5 compares baseline loaders at a 20 GB cache budget with CellPack without a cache. On the RAID-0, Lance is the fastest baseline at 77.8K cells/s; BioNeMo, HF Datasets and memmap deliver 26.5–33.1K, MappedCollection 11.9K, Zarr and Parquet 2.7–3.5K, and AnnLoader 1.1K.

Read amplification and caching. Memory-mapped readers lose most of their throughput to read-ahead: with a 256 KiB window they read 386–468 KB per delivered cell, 20–26 times the stored cell, and deliver 8.5–12.3K cells/s. Read-ahead is enabled by default in these loaders, and we keep that default. Across cache budgets up to 80 GB, most baselines improve by at most 30%. Memory-mapped readers gain 1.5–1.7 times only with a cache four times the compressed corpus size.

Table 2. **Bandwidth shifts with increasing cache budgets.** Gray shading marks bottlenecks inferred from the resource profiles.

	Lance					BioNeMo				
Cache	0	2	10	50	100	0	2	10	50	100
SSD	2.40	2.39	2.38	2.45	2.48	–	0.75	0.75	0.74	0.73
IOPS	167.7	166.5	166.3	165.3	163.1	–	182.0	183.2	181.6	179.4
DRAM	56.96	56.82	56.70	57.43	58.10	–	12.15	12.36	12.42	12.56
CPU	14.62	14.66	14.66	14.80	14.85	–	3.48	3.39	3.51	3.73
Out.	0.69	0.68	0.68	0.69	0.70	–	0.26	0.27	0.28	0.29

	Zarr					annbatch*				
Cache	0	2	10	50	100	0	2	10	50	100
SSD	1.51	1.54	1.54	1.32	0.95	0.58	0.57	0.57	0.54	0.50
IOPS	15.3	13.8	13.4	12.6	11.0	2.4	2.3	2.3	2.2	2.1
DRAM	19.26	24.51	24.57	24.38	23.67	14.15	13.97	14.04	13.86	13.78
CPU	13.43	13.89	13.95	14.09	14.24	12.32	12.30	12.30	12.34	12.43
Out.	0.03	0.03	0.03	0.03	0.03	0.53	0.53	0.53	0.52	0.53

	CellPack								
Cache	0	2	10	50	100	200	400	800	Peak
SSD	6.02	5.90	5.60	2.62	0.00	0.00	0.04	0.00	6.00
IOPS	1,365.3	1,334.7	1,260.8	591.0	0.0	0.0	9.6	0.0	1,054
DRAM	23.94	23.14	28.91	36.00	38.75	40.64	58.22	89.83	178.56
CPU	15.78	16.58	18.96	21.61	21.85	21.78	21.49	19.66	16
Out.	11.18	10.93	11.97	15.39	17.36	17.36	18.84	28.25	–

SSD/DRAM/Out.: GB/s; IOPS: k/s; CPU: cores. CellPack: 32-CPU quota; baselines: 16 physical cores. Peak refers to the original SSD.

Storage bandwidth. Doubling the devices changes each baseline by about 5% or less at 20 GB, except TileDB (+12%). Apart from memory-mapped readers with read-ahead, no baseline reads more than 3.1 GB/s, so on 16 cores per-cell processing limits throughput.

CellPack. Without a cache, 16 workers deliver 1.165M cells/s on the single SSD and 1.365M on RAID-0; 32 workers deliver 1.365M and 2.520M, respectively. With 32 cores and a 20 GB budget, online planning reaches 2.914M and 2.904M, and offline planning 3.596M and 3.543M. Raising the offline budget to 80 GB yields 3.603M and 3.580M.

Storage and worker capacity must scale together: doubling workers raises no-cache throughput by 17% on one SSD but 85% on RAID-0. Likewise, the second SSD adds 17% at 16 workers and 85% at 32. With a 20 GB cache, both devices deliver nearly identical rates within each planning mode, consistent with reuse removing much of the storage constraint.

Where the bottleneck moves. Table 2 reports device reads, DRAM traffic and output delivery. Without a cache, 32 workers deliver 1.365M cells/s, read 6.02 GB/s at 1.365M IOPS and supply 11.18 GB/s of output on 15.78 cores, about one device request per delivered cell. A 100% budget removes device reads and delivers 17.36 GB/s (2.120M cells/s) on 21.85 cores. At 800%, offline page-and-row planning

Table 3. **Compression–throughput trade-offs.** Higher values are better for all metrics.

Method	Ratio (×)	Enc. (MB/s)	Dec. (MB/s)
Codecs and containers			
Snappy	1.60	619.2	1,559.1
LZF	1.68	245.7	676.1
LZ4	1.69	468.1	3,021.3
LZ4HC (12)	2.64	6.3	3,071.5
Gzip (6)	2.65	24.3	260.0
Zlib (12)	2.87	3.9	780.7
Zstd (3, default)	3.00	242.8	898.2
Blosc1 (Zstd-9)	4.04	2.8	1,052.3
Zstd (22)	4.48	1.7	980.9
Brotli (11)	4.68	0.5	378.0
Zarr (PCodec-8)	4.79	124.3	669.0
Bzip2 (9)	4.81	11.2	31.3
PCodec (12)	4.82	29.9	912.3
XZ (9)	4.88	1.6	89.5
Entropy coders			
Huffman (byte)	1.45	222.9	287.6
Arithmetic (byte)	1.61	70.8	26.7
rANS (order 0, word)	1.95	97.3	522.1
CellPack	5.89	347.2	768.5

delivers 3.448M cells/s and 28.25 GB/s, and DRAM traffic reaches 89.83 GB/s against a 178.56 GB/s memory reference. These resource profiles place CellPack's bottleneck at the SSD for 0–10% budgets, the CPU for 50–100%, and memory bandwidth for 200–800%. Lance's profile indicates a memory-bandwidth bottleneck across the tested budgets.

Output bandwidth counts dense model inputs, not compressed bytes read from disk. At no cache, 11.18 GB/s of delivered output accompanies 6.02 GB/s of device reads; the difference reflects decompression and expansion into the dense panel. Once page reads disappear, decoding, gene mapping and output writes remain. This explains why fitting the compressed corpus in memory does not eliminate the entire loading cost.

5.3 Compression and decoding cost

Protocol. Table 3 reports mean compression ratios and single-core encoding and decoding speeds, with equal weight for each raw or normalised dataset matrix. Speeds are measured in uncompressed bytes per second. For each codec family, we report the configuration with the highest pooled compression ratio; CellPack uses 4,096-cell chunks. Indices use 16 or 32 bits, and values retain the source dtype.

CellPack uses 432 raw and 302 normalised blocks, each in three independent processes. Raw source-order prefixes contain at most 100 MiB of indices and values; normalised blocks use the same cells. General codecs use 428 raw and 296 normalised seeded prefixes. Figure 3b separately compares unit sizes on fixed inputs.

Results. CellPack reaches 5.89× compression, encodes at 347.2 MB/s and decodes at 768.5 MB/s. XZ (9) and PCodec (12) achieve 4.88× and 4.82×, but decode at 89.5 and 912.3 MB/s, respectively. Zstd (22) improves on default Zstd's ratio from 3.00× to 4.48× while reducing encoding speed from 242.8 to 1.7 MB/s, and LZ4 decodes at 3,021.3 MB/s with a ratio of only 1.69×. The CellPack corpus occupies 20.340 GB, or 2.041 kB/cell; cell records take 1.918 kB/cell and span 1.076 4 KiB pages on average.

Why ratio and sequential decode speed are insufficient. Raw and normalised CellPack blocks compress by 5.07× and 7.07×, respectively, supporting the observation that floating-point normalisation preserves repeated values within cells. LZ4 decodes a contiguous byte stream faster, but its larger representation requires more storage traffic. Conversely, a compact block codec may decode many unrequested cells for one random request. The independent record and near-one-page footprint connect compression to the actual cost of delivering a cell, which the sequential codec rates alone do not capture.

5.4 How memory changes the bottleneck

Setup. This cache-policy ablation runs within the CellPack loader. One NVMe SSD feeds 32 loader workers, with 16 additional cores for online planning. Four shuffled epochs give 39.87M accesses; measurements cover epochs 2–4. Budgets are fractions of the compressed page bytes. The policies are global-lock LRU, 64-shard CLOCK, ARC, S3-FIFO, W-TinyLFU, static top-k, and offline and online CellPack plans.

Fewer misses at the same budget. Under random order, recency and frequency carry little information: every cell is used once per epoch, and pages are shared by unrelated cells. At a 10% budget, LRU and CLOCK remove 9.1–9.3% of page misses; ARC, S3-FIFO and W-TinyLFU remove 13.4–18.0%, and static top-k 23.5%. CellPack's page plan removes 38.4%. At a 1% budget it removes 2.8× the misses of the best baseline (Figure 6a).

From misses to throughput. Without a cache the loader delivers 1.45M cells/s while reading 6.4 GB/s. The offline page plan reaches 2.29M cells/s at a 10% budget, 3.63M at 50% and 4.06M at 100%, gains of 58%, 150% and 180%. At 10%, static top-k reaches 1.87M cells/s and ARC, S3-FIFO and W-TinyLFU 1.69–1.76M, while global-lock LRU falls to 0.80M, 0.55× no cache.

For a storage-limited path, 38.4% fewer misses predict 1.62× throughput if the cost per page read is unchanged. The measured 1.58× gain at 10% is close to this simple

(a) Page misses removed (%)

Budget	1%	2%	5%	10%	20%	50%
CellPack	**12.9** +15%	**18.1** +22%	**28.0** +37%	**38.4** +58%	**52.8** +106%	**77.7** +150%
CellPack*	**12.9** +14%	**18.1** +20%	**28.0** +36%	**38.5** +56%	**52.8** +103%	**77.7** +145%
Static top-k	4.6 +4%	7.5 +7%	14.4 +17%	23.5 +29%	37.4 +57%	67.9 +122%
W-TinyLFU	2.8 +3%	5.0 +6%	10.4 +12%	18.0 +22%	31.3 +44%	63.1 +114%
S3-FIFO	3.0 +4%	5.0 +6%	9.9 +11%	16.0 +19%	25.3 +33%	49.3 +91%
ARC	2.5 +2%	4.2 +4%	8.2 +8%	13.4 +16%	19.3 +22%	44.5 +78%
CLOCK	1.0 +2%	1.9 +3%	4.7 +5%	9.3 +11%	18.4 +22%	45.5 +79%
LRU	0.9 -33%	1.9 -36%	4.6 -41%	9.1 -45%	18.0 -47%	45.0 -47%
Belady	*13.3*	*18.5*	*28.3*	*38.7*	*53.0*	*77.8*

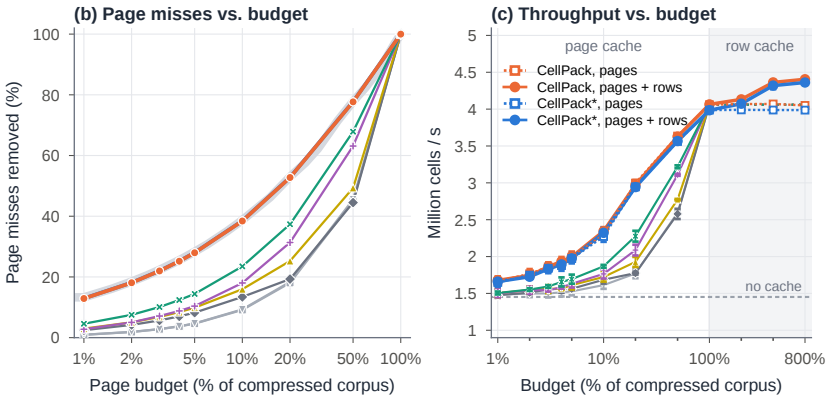


Figure 6. **Cache-policy ablation in the CellPack loader: page-miss reduction and throughput.** CellPack and CellPack* use offline and online compilation, respectively. *Pages + rows* additionally retains decoded cells. Budgets are relative to compressed corpus size; gray percentages in (a) give throughput gains over no cache. Points: three-run means; error bars: one sample standard deviation. Dotted extensions mark saturated page-cache references. Belady (gray) is the simulated offline-optimal page cache, the theoretical upper bound.

expectation. Miss reduction is nevertheless insufficient by itself: LRU saves some reads but reduces throughput, consistent with the overhead of its globally locked implementation. Keeping the format, trace and loader fixed in this ablation exposes the combined effects of reuse decisions and their execution cost.

Memory beyond the compressed corpus. Once every page fits, a page-only cache saturates: the offline page plan delivers 4.05–4.07M cells/s at budgets of 1–8× the corpus (Figure 6c). Caching decoded rows continues to pay off. The offline pages-and-rows plan reaches 4.41M cells/s at 8× the corpus, 3.03× no cache, with 64% of accesses served from decoded rows; the online plan reaches 4.07M cells/s at 2× and 4.36M at 8×, 3.00× no cache.

At 8×, decoded rows add about 9% over the saturated page-only rate, despite serving 64% of accesses. Most of the gain over no cache therefore comes from avoiding storage reads; row reuse removes decoding but still incurs cache-memory traffic and output construction. The resource profiles provide the complementary evidence for this shift in the limiting work.

5.5 Scaling to one billion cells

Setup. The corpus holds one billion cells (1.85 TB) on the RAID-0 array, with the same fixed 2,048-gene `float32` output. Eight planner cores and 48 loader cores serve a 1 TB compressed-page cache, with each requested cell decoded before delivery. Each epoch uses a fresh global permutation. The online plan sees one billion accesses ahead, and cached pages persist across epochs.

Throughput over time. From loader start, CellPack delivers 5B cells in 25.7 minutes, averaging 3.24M cells/s across start-up and all epochs (Figure 7). Metadata loading precedes this interval and takes 244 s; including it gives 29.8 minutes and 2.80M cells/s. Online planning delivers the first batch 106 s after the loader starts. The first epoch begins with an empty cache, averages a 58% page hit rate and 2.20M cells/s, and reaches peak device reads of 7.1 GB/s. In epochs 2–5, the hit rate rises to 81% and reads fall from 1.80 to 0.80 TB per epoch. Throughput reaches 3.61–3.71M cells/s, averaging 3.67M. Device reads stay below 40% of the array's page-read ceiling as page reuse shifts the bottleneck away from storage. Peak resident memory is 1.29 TB, including a 1.00 TB cache.

Sustained delivery beyond memory. The 1 TB cache holds about 54% of the compressed corpus, yet page sharing and planned reuse yield an 81% hit rate after the first epoch. Epochs 2–5 each complete in 270–277 s, indicating sustained service across fresh global permutations. The run records five billion decodes, zero decoded-row hits and 40.96 TB of output writes. Thus the steady-state rate includes decoding and dense input construction, rather than repeated delivery of precomputed batches. Resident memory exceeds the cache budget because metadata, planning state and execution buffers also occupy memory.

5.6 Relating loader supply to model demand

Training-step throughput measures the cell-delivery rate needed to keep each model supplied. The ten evaluated models have 51K–143M parameters and use several input representations. scVI processes 263K cells/s; CPA reaches 409K on Tahoe and 292K on Replogle. CellOT, CellFlow,



Figure 7. **Loading one billion cells for five epochs from a two-SSD RAID-0 with a 1 TB cache.** Online planning uses a one-billion-cell horizon and retains the cache across epochs. Per-second curves are smoothed over 5 s. Throughput is normalised to its 5 s peak (4.0M cells/s), device reads to the array's page-read ceiling (10.6 GB/s), and resident memory to the memory limit (1.89 TB); page hit rate is shown directly. Right-hand labels give fifth-epoch means and peak memory.

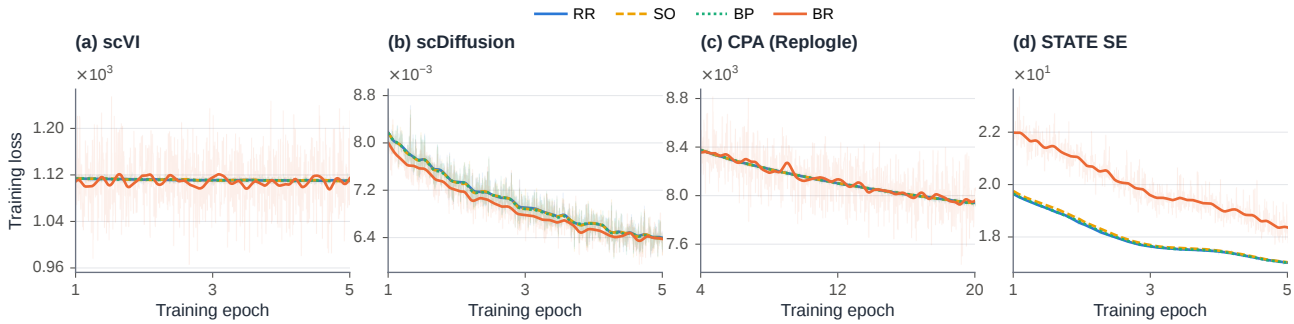


Figure 8. **Training loss under different sampling strategies.** Four model–dataset combinations compare a fresh global permutation each epoch (RR), a repeated global permutation (SO), block sampling after global preshuffling (BP), and block sampling in source order (BR).

STATE ST and scDiffusion process 574K, 424K, 228K and 152K cells/s, respectively. scGPT, STATE SE and UCE remain below 1,000 cells/s, while GEARS reaches 3.34K.

5.7 Effect of sampling order on training

Orders. RR draws a new global permutation each epoch; SO repeats the first permutation. BP shuffles 512-cell blocks of a globally preshuffled sequence, and BR applies annbatch's sampler, with the same block size, to the stored sequence; both use a 32-block buffer. CellPack delivers cells in each specified order. Figure 8 shows training trajectories for four model–dataset combinations. Periodic evaluations use fixed in-distribution (ID) and out-of-distribution (OOD) validation subsets; Table 4 reports the last evaluation.

Within each model–dataset comparison, orders share the training set, initialization and hyperparameters; each epoch covers the same cells once. Validation uses fixed inputs and isolated random-number state. BP and BR use the same block and buffer sizes, so their comparison tests the effect of preshuffling rather than a change in the amount of data fetched together.

Training trajectories. BR produces recurrent training-loss excursions in scVI and CPA on Replogle. In STATE SE, BR has persistently higher smoothed training loss than RR, SO and BP. In scDiffusion, BR often has lower training loss than RR.

Validation loss. BR has higher ID and OOD loss than RR at every evaluation from epoch 1 onwards for scVI and scDiffusion, and from epoch 4 onwards for CPA on Replogle. At the last evaluation, its OOD loss is 0.84%, 0.67% and 0.18% higher, respectively. STATE SE ends with BR loss 1.85% higher on ID and 1.56% higher on OOD. SO and BP yield similar losses to RR in these four model–dataset comparisons.

Dependence on the model. The pattern is not universal. Repeated runs for CellFlow and CellOT give BR OOD-loss changes of $0.09 \pm 0.26\%$ and $0.56 \pm 2.09\%$, respectively (mean and sample standard deviation). Both sample within condition-specific pools, limiting exposure to corpus-wide

Table 4. **Validation loss under different sampling strategies.** SO, BP and BR report percentage changes in loss relative to RR.

			Change from RR (%)		
Model	Split	RR loss	SO	BP	BR
scGPT	ID	1.41×10^{2}	+0.03	+0.02	+0.12
	OOD	1.47×10^{2}	+0.05	0.00	+0.11
STATE SE	ID	1.63×10^{1}	−0.03	−0.24	+1.85
	OOD	1.63×10^{1}	−0.04	−0.24	+1.56
STATE ST	ID	1.89×10^{-2}	+11.26	+4.56	+0.90
	OOD	2.50×10^{-1}	+14.75	+0.27	+28.87
GEARS	ID	6.36×10^{-1}	0.00	0.00	0.00
	OOD	6.37×10^{-1}	0.00	0.00	0.00
scVI	ID	1.10×10^{3}	−0.02	−0.02	+1.17
	OOD	1.17×10^{3}	−0.01	0.00	+0.84
CPA (Tahoe)	ID	2.59×10^{2}	+0.15	+0.27	+0.12
	OOD	2.48×10^{2}	+0.38	+0.02	−0.07
CPA (Replogle)	ID	7.93×10^{3}	−0.01	−0.01	+0.18
	OOD	7.92×10^{3}	−0.01	−0.01	+0.18
CellOT	ID	2.05×10^{-3}	+0.55	+0.08	+0.53
	OOD	2.06×10^{-3}	+0.17	+0.56	+2.17
CellFlow	ID	3.06×10^{-1}	−0.02	−0.02	−0.02
	OOD	3.15×10^{-1}	+0.42	+0.13	+0.47
scDiffusion	ID	6.22×10^{-3}	+0.19	+0.10	+0.72
	OOD	6.40×10^{-3}	+0.04	+0.19	+0.67

source structure. The contrasting outcomes motivate preserving the trainer's chosen order. Lower scDiffusion training loss under BR alongside higher validation loss further shows why training loss alone cannot establish better model quality.

6 Related Work

Single-cell loading. AnnData and Scanpy provide the H5AD-based analysis ecosystem (Virshup et al., 2024; Wolf et al., 2018). MappedCollection indexes collections of H5AD files, while scDataset, annbatch, TileDB-SOMA-ML and cellarium organise fetching around blocks, chunks or shards (Lamin Labs, n.d.; D'Ascenzo & Cultrera di Montesano, 2026; Gold et al., 2026; TileDB-SOMA-ML Developers, n.d.; Cellarium AI, n.d.). BioNeMo uses memory-mapped sparse arrays (St. John et al., 2024). These systems make different trade-offs between compression, locality and sampling. CellPack combines independent compressed records with planned reuse of pages and decoded rows.

Training formats and pipelines. FFCV and Progressive Compressed Records adapt image storage to training access (Leclerc et al., 2023; Kuchnik et al., 2021); Youmu studies fine-grained access and shuffling for language-model data (Zhong et al., 2025). Parquet, Lance, HF Datasets and Zarr with TensorStore provide general storage mechanisms (The Apache Software Foundation, n.d.; Lance Developers, n.d.; Lhoest et al., 2021; Zarr Developers, n.d.; Google, n.d.). CellPack specialises record coding for sparse numeric rows. General pipeline systems, including tf.data, Plumber and CoorDL, optimise execution and caching (Murray et al., 2021; Kuchnik et al., 2022; Mohan et al., 2021); SALIENT and RecD exploit graph and recommendation workloads (Kaler et al., 2022; Zhao et al., 2023).

Caching with future knowledge. Belady's rule is optimal for equal-sized items under its replacement model (Belady, 1966); variable sizes and costs make offline caching harder (Chrobak et al., 2012). Hawkeye learns from Belady decisions (Jain & Lin, 2016), while Quiver, GNNLab and Marius use workload structure to manage training data (Kumar & Sivathanu, 2020; Yang et al., 2022; Mohoney et al., 2021). Seneca divides memory among encoded, decoded and augmented images (Desai et al., 2026). CellPack plans representation lifetimes from a supplied access sequence and compares their saved work through a machine cost model.

7 Conclusion

CellPack organises data loading around the cell access requested by training. Independent compressed records reduce the work of a random read, asynchronous workers construct batches in the specified order, and planned reuse trades memory against storage and decoding costs. Together, these mechanisms reduce per-cell work and sustain batch delivery from storage.

REFERENCES

Arc Institute. Arc Institute announced as new grantee in The Audacious Project's 2025 cohort. Arc Institute, February 2026. URL https://arcinstitute.org/news/the-audacious-project-2025. Published February 24, 2026. Accessed October 31, 2026.

Axboe, J. Efficient IO with io_uring. Technical report, 2019. URL https://kernel.dk/io_uring.pdf. Version 0.4, October 15, 2019. Accessed October 31, 2026.

Belady, L. A. A study of replacement algorithms for a virtual-storage computer. *IBM Systems Journal*, 5(2): 78–101, 1966. doi: 10.1147/sj.52.0078.

Cellarium AI. cellarium-ml: distributed single-cell data analysis. Software and documentation, n.d. URL https://github.com/cellarium-ai/cellarium-ml. Accessed October 31, 2026.

Chrobak, M., Woeginger, G. J., Makino, K., and Xu, H. Caching is hard—even in the fault model. *Algorithmica*, 63(4):781–794, 2012. doi: 10.1007/s00453-011-9502-9.

Cui, H., Wang, C., Maan, H., Pang, K., Luo, F., Duan, N., and Wang, B. scGPT: toward building a foundation model for single-cell multi-omics using generative AI. *Nature Methods*, 21:1470–1480, 2024. doi: 10.1038/s41592-024-02201-0.

CZI Cell Science Program et al. CZ CELLxGENE Discover: a single-cell data platform for scalable exploration, analysis and modeling of aggregated data. *Nucleic Acids Research*, 53(D1):D886–D900, 2025. doi: 10.1093/nar/gkae1142. URL https://academic.oup.com/nar/article/53/D1/D886/7912032.

D'Ascenzo, D. and Cultrera di Montesano, S. scDataset: Scalable data loading for deep learning on large-scale single-cell omics. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pp. 22246–22262. PMLR, 2026. URL https://proceedings.mlr.press/v306/d-ascenzo26a.html.

Desai, O., Jiao, Z., Pei, S., Bhimani, J., and Kim, B. S. Preparation meets opportunity: Enhancing data preprocessing for ML training with Seneca. In *24th USENIX Conference on File and Storage Technologies (FAST)*, pp. 221–238, 2026. URL https://www.usenix.org/conference/fast26/presentation/desai.

Duda, J. Asymmetric numeral systems: entropy coding combining speed of Huffman coding with compression rate of arithmetic coding. *arXiv preprint arXiv:1311.2540*, 2014. doi: 10.48550/arXiv.1311.2540. URL https://arxiv.org/abs/1311.2540v2.

Gandhi, S., Javadi, F., Svensson, V., Khan, U., Jones, M. G., Yu, J., Merico, D., Goodarzi, H., and Alidoust, N. Tahoe-x1: Scaling perturbation-trained single-cell foundation models to 3 billion parameters. *bioRxiv*, 2025. doi: 10.1101/2025.10.23.683759. URL https://www.biorxiv.org/content/10.1101/2025.10.23.683759v1. Preprint.

Gold, I., Fischer, F., Arnoldt, L., Wolf, F. A., and Theis, F. J. annbatch unlocks terabyte-scale training of biological data in anndata. *arXiv preprint arXiv:2604.01949*, 2026. doi: 10.48550/arXiv.2604.01949. URL https://arxiv.org/abs/2604.01949v2.

Gong, J., Hao, M., Cheng, X., Zeng, X., Liu, C., Ma, J., Zhang, X., Wang, T., and Song, L. xTrimoGene: An efficient and scalable representation learner for single-cell RNA-Seq data. In *Advances in Neural Information Processing Systems*, volume 36, pp. 69391–69403, 2023. doi: 10.52202/075280-3039. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/db68f1c25678f72561ab7c97ce15d912-Abstract-Conference.html.

Google. TensorStore: a library for reading and writing large multi-dimensional arrays. Software and documentation, n.d. URL https://github.com/google/tensorstore. Accessed October 31, 2026.

Hugging Face. Process: Sort, shuffle, select, split, and shard. Datasets documentation, n.d. URL https://huggingface.co/docs/datasets/process. Accessed October 31, 2026.

Jain, A. and Lin, C. Back to the future: Leveraging Belady's algorithm for improved cache replacement. In *Proceedings of the 43rd International Symposium on Computer Architecture (ISCA)*, pp. 78–89, 2016. doi: 10.1109/ISCA.2016.17.

Kaler, T., Stathas, N., Ouyang, A., Iliopoulos, A.-S., Schardl, T. B., Leiserson, C. E., and Chen, J. Accelerating training and inference of graph neural networks with fast sampling and pipelining. In *Proceedings of Machine Learning and Systems (MLSys)*, volume 4, pp. 172–189, 2022. URL https://proceedings.mlsys.org/paper_files/paper/2022/file/afacc5db3e0e85b446e6c7727cd7dca5-Paper.pdf.

Kuchnik, M., Amvrosiadis, G., and Smith, V. Progressive compressed records: Taking a byte out of deep learning data. *Proceedings of the VLDB Endowment*, 14(11):2627–2641, 2021. doi: 10.14778/3476249.3476308.

Kuchnik, M., Klimovic, A., Šimša, J., Smith, V., and Amvrosiadis, G. Plumber: Diagnosing and removing performance bottlenecks in machine learning data

pipelines. In *Proceedings of Machine Learning and Systems (MLSys)*, volume 4, pp. 33–51, 2022. URL https://proceedings.mlsys.org/paper_files/paper/2022/file/d0e90e9a9310570dfa643aa3b2da6e89-Paper.pdf.

Kumar, A. V. and Sivathanu, M. Quiver: An informed storage cache for deep learning. In *18th USENIX Conference on File and Storage Technologies (FAST)*, pp. 283–296, 2020. URL https://www.usenix.org/conference/fast20/presentation/kumar.

Lamin Labs. LaminDB: a data framework for biology. Software and documentation, n.d. URL https://github.com/laminlabs/lamindb. Accessed October 31, 2026.

Lance Developers. Lance: a modern columnar data format for ML. Software and documentation, n.d. URL https://github.com/lance-format/lance. Accessed October 31, 2026.

Leclerc, G., Ilyas, A., Engstrom, L., Park, S. M., Salman, H., and Madry, A. FFCV: Accelerating training by removing data bottlenecks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 12011–12020, 2023. doi: 10.1109/CVPR52729.2023.01156.

Lhoest, Q. et al. Datasets: A community library for natural language processing. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 175–184, 2021. doi: 10.18653/v1/2021.emnlp-demo.21. URL https://aclanthology.org/2021.emnlp-demo.21/.

Mohan, J., Phanishayee, A., Raniwala, A., and Chidambaram, V. Analyzing and mitigating data stalls in DNN training. *Proceedings of the VLDB Endowment*, 14 (5):771–784, 2021. doi: 10.14778/3446095.3446100.

Mohoney, J., Waleffe, R., Xu, H., Rekatsinas, T., and Venkataraman, S. Marius: Learning massive graph embeddings on a single machine. In *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pp. 533–549, 2021. URL https://www.usenix.org/conference/osdi21/presentation/mohoney.

Murray, D. G., Šimša, J., Klimovic, A., and Indyk, I. tf.data: A machine learning data processing framework. *Proceedings of the VLDB Endowment*, 14(12):2945–2958, 2021. doi: 10.14778/3476311.3476374.

NVIDIA. Introduction to NVIDIA DGX H100/H200 systems. NVIDIA DGX H100/H200 User Guide, n.d. URL https://docs.nvidia.com/dgx/dgxh100-user-guide/introduction-to-dgxh100.html. Accessed October 31, 2026.

St. John, P. et al. BioNeMo framework: a modular, high-performance library for AI model development in drug discovery. *arXiv preprint arXiv:2411.10548*, 2024. doi: 10.48550/arXiv.2411.10548. URL https://arxiv.org/abs/2411.10548v1.

The Apache Software Foundation. Apache Parquet. Software and documentation, n.d. URL https://parquet.apache.org. Accessed October 31, 2026.

The Linux Kernel Development Community. Page cache. The Linux Kernel documentation, n.d. URL https://docs.kernel.org/mm/page_cache.html. Accessed October 31, 2026.

TileDB, Inc. TileDB-SOMA: a TileDB implementation of the SOMA data model. Software and documentation, n.d. URL https://github.com/single-cell-data/TileDB-SOMA. Accessed October 31, 2026.

TileDB-SOMA-ML Developers. TileDB-SOMA-ML: ExperimentDataset. Software documentation, n.d. URL https://single-cell-data.github.io/TileDB-SOMA-ML/. Accessed October 31, 2026.

Virshup, I., Rybakov, S., Theis, F. J., Angerer, P., and Wolf, F. A. anndata: Access and store annotated data matrices. *Journal of Open Source Software*, 9(101):4371, 2024. doi: 10.21105/joss.04371.

Wolf, F. A., Angerer, P., and Theis, F. J. SCANPY: large-scale single-cell gene expression data analysis. *Genome Biology*, 19:15, 2018. doi: 10.1186/s13059-017-1382-0.

Yang, J., Tang, D., Song, X., Wang, L., Yin, Q., Chen, R., Yu, W., and Zhou, J. GNNLab: A factored system for sample-based GNN training over GPUs. In *Proceedings of the Seventeenth European Conference on Computer Systems (EuroSys)*, pp. 417–434, 2022. doi: 10.1145/3492321.3519557.

Youngblut, N. D., Carpenter, C., Nayebnazar, A., Adduri, A., Shah, R., Ricci-Tam, C., Prashar, J., Ilango, R., Teyssier, N., Konermann, S., Hsu, P. D., Dobin, A., Burke, D. P., Goodarzi, H., and Roohani, Y. H. scBaseCount: An AI agent-curated, standardized, auto-updated single-cell data repository. *Cell*, 189(19):5932–5944.e6, 2026. doi: 10.1016/j.cell.2026.08.025. URL https://doi.org/10.1016/j.cell.2026.08.025.

Zarr Developers. Zarr: chunked, compressed, N-dimensional arrays. Software and documentation, n.d. URL https://zarr.dev. Accessed October 31, 2026.

Zhao, M., Choudhary, D., Tyagi, D., Somani, A., Kaplan, M., Lin, S.-H., Pumma, S., Park, J., Basant, A., Agarwal, N., Wu, C.-J., and Kozyrakis, C. RecD: Deduplication for end-to-end deep learning recommendation model training infrastructure. In *Proceedings of Machine Learning and Systems (MLSys)*, volume 5, pp. 754–767, 2023. URL https://proceedings.mlsys.org/paper_files/paper/2023/file/f9b15fec25182f2d70af68a39546d60e-Paper-mlsys2023.pdf.

Zheng, G. X. Y., Terry, J. M., Belgrader, P., et al. Massively parallel digital transcriptional profiling of single cells. *Nature Communications*, 8:14049, 2017. doi: 10.1038/ncomms14049.

Zhong, T., Zhao, J., Su, Q., and Fox, G. Youmu: Efficient columnar data pipeline for LLM training. In *Proceedings of Machine Learning and Systems (MLSys)*, volume 7, 2025. URL https://proceedings.mlsys.org/paper_files/paper/2025/file/136b9a13861308c8948cd308ccd02658-Paper-Conference.pdf.